



KALASALINGAM
ACADEMY OF RESEARCH AND EDUCATION
(DEEMED TO BE UNIVERSITY)

Under sec. 3 of UGC Act 1956. Accredited by NAAC with "A" Grade



JAVA PROGRAMMING

BY

Dr.T.Dhiliphan Rajkumar

Course Outline



CO 1. Understand the object oriented concepts using java



CO 2. Apply Object Oriented Concepts to develop Java Application



CO 3. Identify, Formulate and Analyze a Real World Problem and Develop a solution using Java Programming Concepts



CO 4. Ability to Work individually as well as in team to solve problem



CO 5. Communicate effectively with technical community during Interviews

Course description

Java still has its importance and popularity in the software industry, in spite of recent development of several high level languages. It has an excellent support of high-level as well as low-level functionality, which makes it suitable for many applications. This course provides students with a comprehensive study of the Java programming language. Instead of Classroom lectures, practical classes stress the strengths of Java, which provide the students with the means of writing efficient, maintainable, and portable code. A course with mini project is one of the great learning opportunities to develop students in various aspects. This course will also help the students during their placement sittings, as most of the companies test proficiency in programming using Java.

UNIT I OOP IN JAVA & INHERITANCE

Object Oriented Programming Concepts - OOP in Java - Characteristics of Java -Fundamental Programming Structures in Java - Defining classes in Java - Comments, Data Types, Variables, Operators, Control Flow, Arrays - constructors, methods -access specifiers - static members - Packages- Inheritance - Super classes- sub classes -Protected members - constructors in sub classes- Strings.

UNIT II EXCEPTION HANDLING AND I/O

Exceptions - exception hierarchy - throwing and catching exceptions - built-in exceptions, creating own exceptions, Stack Trace Elements-Input / Output Basics - Streams - Byte streams and Character streams - Reading and Writing Console - Reading and Writing Files - abstract classes and methods - final methods and classes

UNIT III INTERFACES AND MULTITHREADING

Interfaces - defining an interface, implementing interface, differences between classes and interfaces - extending interfaces - Object cloning - inner classes-Differences between multi-threading and multitasking, thread life cycle, creating threads, synchronizing threads, Inter-thread communication, daemon threads - Generic Programming.

UNIT IV AWT AND EVENT DRIVEN PROGRAMMING

AWT Event Hierarchy- Components - Graphics programming - Applets-Frame -working with 2D shapes - Using color, fonts, and images - Basics of event handling - event handlers - adapter classes - actions - mouse events - Introduction to Swing - layout management - Swing Components - Windows -Menus - Dialog Boxes.

UNIT V NETWORKING AND JDBC

Networking Basics - The Networking Classes and Interfaces -TCP/IP Client Sockets URL TCP/IP Server Sockets - **Datagrams** - A Relational Database Overview - JDBC Introduction - JDBC Product Components - JDBC Architecture - Case studies.



Unit II

Outcomes

- Handle exceptions
- Create own Exception handlers
- Get Input from command line
- Read/Write I/O using files
- Use Abstract and Final Methods and Classes

Unit 2 Outline



Lesson 1. Exceptions



Lesson 2. Built In and Custom Exceptions



Lesson 3. Byte Stream



Lesson 4. Character Stream



Lesson 5. Abstract and Final

Describes the exception handling mechanisms, different types of exceptions and their hierarchy, way to obtain command line arguments as inputs both numbers and characters, programming for files and abstract & final keywords.

Course Progress (use before starting new lesson)

Lesson 1. Exceptions

Lesson 2. Built in and Custom Exceptions

Lesson 3. Byte Stream

Lesson 4. Character Stream

Lesson 5. Abstract and Final

Motivation

- We seek robust programs
- When something unexpected occurs
 - Ensure program detects the problem
 - Then program must do something about it
- Extensive testing of special situations can result in "spaghetti code"
- Need mechanism to check for problem where it could occur
- When condition does occur
 - Have control passed to code to handle the problem

Exceptions

The exception handling in java is one of the powerful mechanism to handle the runtime errors so that normal flow of the application can be maintained.

An exception is an event, which occurs during the execution of a program that interrupts the normal flow of the program. It is an error thrown by a class or method reporting an error in code.

The process of converting system error messages into user friendly error message is known as **Exception handling**.

This is one of the powerful features of Java to handle run time error and maintain normal flow of java application.



Exception and Exception handling

Exception

Exception is an abnormal condition. In java, exception is an event that disrupts the normal flow of the program. It is an object which is thrown at runtime.

Exception handling

Exception Handling is a mechanism to handle runtime errors such as ClassNotFoundException, IO, SQL, Remote etc.

Advantage of Exception Handling

The core advantage of exception handling is to maintain the normal flow of the **application**. Exception normally disrupts the normal flow of the application that is why we use exception handling. Let's take a scenario:

statement 1;

statement 2; //exception occurs

statement 3;

statement 4;

statement 5;

Suppose there is 5 statements in your program and there occurs an exception at statement 2, rest of the code will not be executed i.e. statement 3 to 5 will not run. If we perform exception handling, rest of the statement will be executed. That is why we use exception handling in java.

When an exception can occur?

Exception can occur in two ways

- at runtime (known as runtime exceptions)
- at compile-time (known as Compile-time exceptions).

Reasons for Exceptions

There can be several reasons for an exception. For example, following situations can cause an exception

- Opening a non-existing file
- Network connection problem
- Operands being manipulated are out of prescribed ranges

Exception Handling

- When an exceptional condition arises, an object representing that exception is created and thrown in the method that caused the error.
- That method may choose to handle the exception itself, or pass it on. Either way, at some point, the exception is caught and processed.
- Java exception handling is managed via five keywords: **try**, **catch**, **throw**, **throws**, and **finally**.
- Program statements that you want to monitor for exceptions are contained within a **try block**. If an exception occurs within the try block, it is thrown.
- Your code can **catch** this exception (using catch) and handle it in some rational manner. System-generated exceptions are automatically thrown by the Java run-time system.

- To manually throw an exception, use the keyword **throw**. Any exception that is thrown out of a method must be specified as such by a **throws** clause.
- Any code that absolutely must be executed after a try block completes is put in a **finally** block.
- Code within try/catch block is referred as protected code. Syntax is as follows

```
try { // Protected code }  
catch (ExceptionName e1)  
    { // Catch block }  
Finally ( // block to be executed at end }
```

Throwing and Catching Exceptions

- The code which is prone to exceptions is placed in the try block.
- When an exception occurs, that exception occurred is handled by catch block associated with it.
- Every try block should be immediately followed either by a catch block or finally block.
- A catch statement involves declaring the type of exception you are trying to catch.
- If an exception occurs in protected code, the catch block (or blocks) that follows the try is checked.
- If the type of exception that occurred is listed in a catch block, the exception is passed to the catch block much as an argument is passed into a method parameter.

Single Catch

```
class Exc2 {  
    public static void main( String args []) { int d, a;  
    try { // monitor a block of code.  
        d = 0;  
        a = 42 / d;  
        System . out. println (" This will not be printed .");  
    }  
    catch ( ArithmeticException e) // divide - by- zero error  
    {  
        System . out. println (" Division by zero.");  
    }  
    System . out. println (" After catch statement.");  
    }  
}
```

Using Multiple Catch

- In some cases, more than one exception could be raised by a single piece of code. To handle this type of situation, you can specify two or more catch clauses, each catching a different type of exception.
- When an exception is thrown, each catch statement is inspected in order, and the first one whose type matches that of the exception is executed.
- After one catch statement executes, the others are bypassed, and execution continues after the try / catch block.

Example of Multiple Catch

```
class MultipleCatches
{
    public static void main( String args [])
    { try {
        int a = args. length ;

        System . out. println (" a  = " + a);  int b = 42 / a;

        int c[] = { 1 };  c[42] = 99;
    }
    catch ( ArithmeticException e)
    {
        System . out. println (" Divide by 0: " + e);
    }

    catch ( Array Index Out OfBounds Exception e)
    {

        System . out. println (" Array index oob: " + e);
    }
    System . out. println (" After try/ catch blocks.");
}
```

Usage of throw

- It is possible for your program to throw an exception explicitly, using the **throw** statement.
- The general form of throw is shown here: **throw ThrowableInstance;**
- Here, ThrowableInstance must be an object of type Throwable or a subclass of Throwable.
- Primitive types, such as int or char, as well as non-Throwable classes, such as String and Object, cannot be used as exceptions.
- The flow of execution stops immediately after the throw statement; any subsequent statements are not executed.
- The nearest enclosing try block is inspected to see if it has a catch statement that matches the type of exception.
- If it does find a match, control is transferred to that statement. If not, then the next enclosing try statement is inspected, and so on.
- If no matching catch is found, then the default exception handler halts the program and prints the stack trace.

Usage of throw - Example

```
class Throw Demo {  
    static void demoproc () { try {  
        throw new NullPointerException (" demo");  
    } catch ( NullPointerException e) { System . out. println (" Caught inside demoproc."); throw  
e; // rethrow the exception  
    }  
}  
    public static void main( String args []) { try {  
        demoproc ();  
    } catch ( NullPointerException e) { System . out. println (" Recaught: " + e);  
    }  
}
```

Usage of throws

- If a method is capable of causing an exception that it does not handle, it must specify this behavior so that callers of the method can guard themselves against that exception.
- You do this by including a throws clause in the method's declaration.
- A throws clause lists the types of exceptions that a method might throw.
- This is necessary for all exceptions, except those of type Error or RuntimeException, or any of their subclasses.
- All other exceptions that a method can throw must be declared in the throws clause.
- If they are not, a compile-time error will result.
- This is the general form of a method declaration that includes a throws clause:

```
type method-name(parameter-list) throws exception-list  
{  
// body of method  
}
```

- Here, exception-list is a comma-separated list of the exceptions that a method can throw.

Usage of throws - Example

```
class ThrowsDemo {  
    static void throw One () throws IllegalAccessException { System . out. println (" Inside throw  
One.");  
    throw new IllegalAccessException (" demo");  
}  
  
    public static void main( String args []) { try {  
        throw One ();  
    } catch ( IllegalAccessException e) { System . out. println (" Caught " + e);  
    }  
    }  
}
```

Usage of finally

- **finally** creates a block of code that will be executed after a try /catch block has completed and before the code following the try/catch block.
- The finally block will execute whether or not an exception is thrown.
- If an exception is thrown, the finally block will execute even if no catch statement matches the exception.

Example of finally



```
class Finally Demo { static void procA () {  
try {  
  
System . out . println (" inside procA "); throw new RuntimeException (" demo");  
} finally { System . out . println (" procA ' s finally ");  
}  
  
static void procB () { try {  
  
System . out . println (" inside procB "); return ;  
} finally { System . out . println (" procB ' s finally ");  
}  
  
public static void main( String args []) { try {  
procA ();  
} catch ( Exception e) { System . out . println (" Exception caught"); procB ();  
} }  
}
```

Topic 1
Exceptions

Topic 2
Exception Handling

Topic 3
Throwing and Catching Exceptions

Topic 4
Multiple Catch

Topic 5
Usage of throw, throws and finally

First Lesson Summary

Learned about Exception handling and their related terms

Course Progress (use before starting new lesson)



Lesson 1. Exceptions

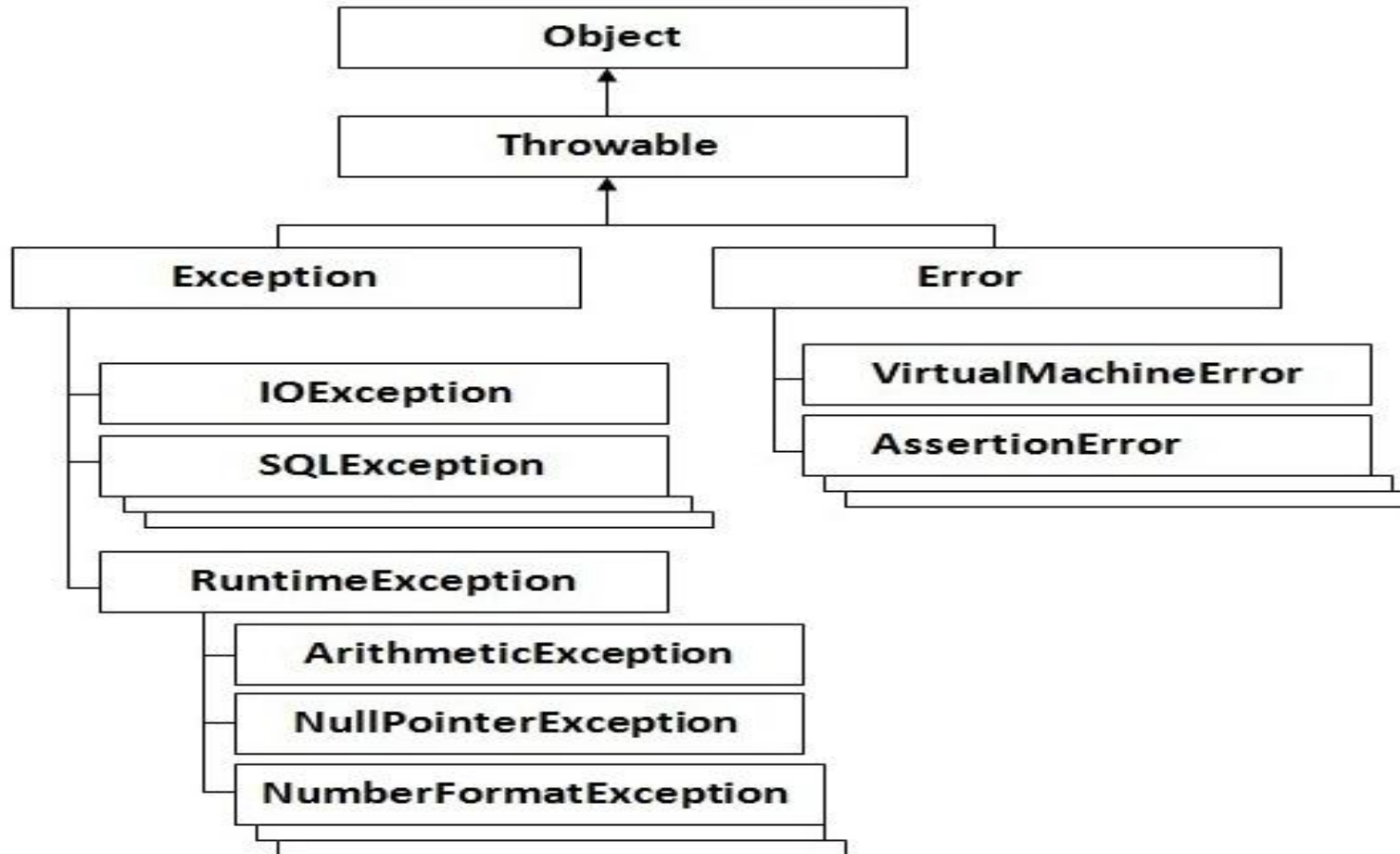
Lesson 2. Built in and Custom Exceptions

Lesson 3. Byte Stream

Lesson 4. Character Stream

Lesson 5. Abstract and Final

Hierarchy of Java Exception classes

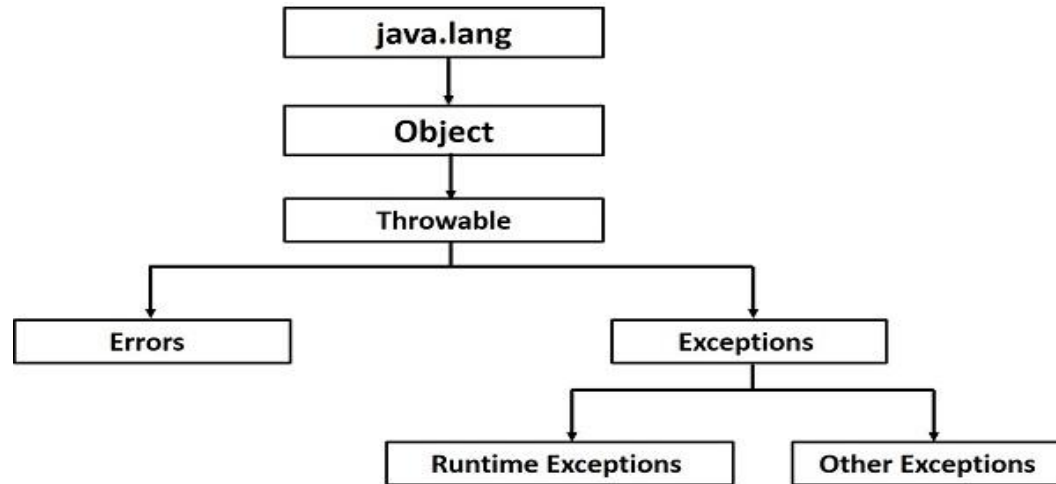


Types of Exceptions

There are mainly two types of exceptions: checked and unchecked where error is considered as unchecked exception. The sun microsystem says there are three types of exceptions:

- Checked Exception
- Unchecked Exception
- Error

Hierarchy of Java Exception



Checked, Unchecked and Error

1) Checked Exception

The classes that extend Throwable class except RuntimeException and Error are known as checked exceptions e.g. IOException, SQLException etc. Checked exceptions are checked at compile-time.

2) Unchecked Exception

The classes that extend RuntimeException are known as unchecked exceptions e.g. ArithmeticException, NullPointerException, ArrayIndexOutOfBoundsException etc. Unchecked exceptions are not checked at compile-time rather they are checked at runtime.

3) Error

Error is irrecoverable e.g. OutOfMemoryError, VirtualMachineError, AssertionError etc.

Common scenarios where exceptions may occur



1) Scenario where ArithmeticException occurs

If we divide any number by zero, there occurs an ArithmeticException.

```
int a=50/0; //ArithmeticException
```

2) Scenario where NullPointerException occurs

If we have null value in any variable, performing any operation by the variable occurs an NullPointerException.

```
String s=null;
```

```
System.out.println(s.length()); //NullPointerException
```

Common scenarios Con”

3) Scenario where NumberFormatException occurs

The wrong formatting of any value, may occur NumberFormatException. Suppose I have a string variable that have characters, converting this variable into digit will occur NumberFormatException.

```
String s="abc";
```

```
int i=Integer.parseInt(s); //NumberFormatException
```

4) Scenario where ArrayIndexOutOfBoundsException occurs

If you are inserting any value in the wrong index, it would result ArrayIndexOutOfBoundsException as shown below:

```
int a[]=new int[5];
```

```
a[10]=50; //ArrayIndexOutOfBoundsException
```

Arithmetic exception

```
class ArithmeticException_Demo {  
    public static void main(String args[])  
    {  
        try {  
            int a = 30, b = 0;  
            int c = a / b; // cannot divide by zero  
            System.out.println("Result = " + c);  
        }  
        catch (ArithmeticException e) {  
            System.out.println("Can't divide a number by 0");  
        }  
    }  
}
```

Output

Can't divide a number by 0

ArrayIndexOutOfBoundsException Exception

```
class ArrayIndexOutOfBounds_Demo {  
    public static void main(String args[])  
    {  
        try {  
            int a[] = new int[5];  
            a[6] = 9; // accessing 7th element in an array of  
                // size 5  
        }  
        catch (ArrayIndexOutOfBoundsException e) {  
            System.out.println("Array Index is Out Of Bounds");  
        }  
    }  
}
```

Output

Array Index is Out Of Bounds

ClassNotFoundException

```
class B
{

}

class G
{

}

class MyClass {
public static void main(String[] args)
{
    Object o = class.forName(args[0]).newInstance();
    System.out.println("Class created for" + o.getClass().getName());
}
}
```

Output

ClassNotFoundException

Other standard Exceptions available

FileNotFoundException

IOException

InterruptedException

NoSuchMethodException

NullPointerException

NumberFormatException

StringIndexOutOfBoundsException

ClassCastException

StackOverflowError

NoClassDefFoundError

IllegalArgumentException

Exceptions Methods



Important Methods available in throwable class

- **public String getMessage()**

Returns a detailed message about the exception that has occurred. This message is initialized in the Throwable constructor.

- **public Throwable getCause()**

Returns the cause of the exception as represented by a Throwable object.

- **public String toString()**

Returns the name of the class concatenated with the result of getMessage().

Exceptions Methods Con”

- **public void printStackTrace()**

Prints the result of toString() along with the stack trace to System.err, the error output stream.

- **public StackTraceElement [] getStackTrace()**

Returns an array containing each element on the stack trace. The element at index 0 represents the top of the call stack, and the last element in the array represents the method at the bottom of the call stack.

- **public Throwable fillInStackTrace()**

Fills the stack trace of this Throwable object with the current stack trace, adding to any previous information in the stack trace.

Exception Hierarchy

All exception types are subclasses of the built-in class **Throwable**. Thus, **Throwable** is at the top of the exception class hierarchy. Immediately below **Throwable** are two subclasses that partition exceptions into two distinct branches.

One branch is headed by **Exception**.

- This class is used for exceptional conditions that user programs should catch.
- This is also the class that you will subclass to create your own custom exception types.
- There is an important subclass of **Exception**, called **RuntimeException**.
- Exceptions of this type are automatically defined for the programs that you write and include things such as division by zero and invalid array indexing.

Exception Hierarchy Con”

The other branch is topped by **Error**, which defines exceptions that are not expected to be caught under normal circumstances by your program.

- Exceptions of type **Error** are used by the Java run-time system to indicate errors having to do with the run-time environment, itself.
- Stack overflow is an example of such an error.

Built in Exceptions

Exception	Meaning
ArithmeticException	Arithmetic error, such as divide-by-zero.
ArrayIndexOutOfBoundsException	Array index is out-of-bounds.
ArrayStoreException	Assignment to an array element of an incompatible type.
ClassCastException	Invalid cast.
EnumConstantNotPresentException	An attempt is made to use an undefined enumeration value.
IllegalArgumentException	Illegal argument used to invoke a method.
IllegalMonitorStateException	Illegal monitor operation, such as waiting on an unlocked thread.
IllegalStateException	Environment or application is in incorrect state.
IllegalThreadStateException	Requested operation not compatible with current thread state.
IndexOutOfBoundsException	Some type of index is out-of-bounds.
NegativeArraySizeException	Array created with a negative size.
NullPointerException	Invalid use of a null reference.
NumberFormatException	Invalid conversion of a string to a numeric format.
SecurityException	Attempt to violate security.
StringIndexOutOfBoundsException	Attempt to index outside the bounds of a string.
TypeNotPresentException	Type not found.
UnsupportedOperationException	An unsupported operation was encountered.

Built in Exceptions

Exception	Meaning
ClassNotFoundException	Class not found.
CloneNotSupportedException	Attempt to clone an object that does not implement the Cloneable interface.
IllegalAccessException	Access to a class is denied.
InstantiationException	Attempt to create an object of an abstract class or interface.
InterruptedException	One thread has been interrupted by another thread.
NoSuchFieldException	A requested field does not exist.
NoSuchMethodException	A requested method does not exist.
ReflectiveOperationException	Superclass of reflection-related exceptions. (Added by JDK 7.)

Custom Exceptions

- To create your own exception types to handle situations specific to your applications: just define a subclass of Exception.
- Your subclasses don't need to actually implement anything—it is their existence in the type system that allows you to use them as exceptions.
- The Exception class does not define any methods of its own.
- It does, of course, inherit those methods provided by Throwable.
- Example while entering a if greater than 10 throws an exception

Custom Exceptions Example

```
class My Exception extends Exception {  
    private int detail;  
    My Exception ( int a) { detail = a; }  
    public String toString () {  
        return " My Exception [" + detail + "]"; }  
}  
class Exception Demo {  
    static void compute( int a) throws My Exception {  
        System . out. println (" Called compute(" + a + ")");  
        if( a > 10) throw new My Exception ( a);  
        System . out. println (" Normal exit");  
    }  
    public static void main( String args []) {  
        try { compute (1); compute (20);}  
        catch ( My Exception e) { System . out. println (" Caught " + e); }  
    }  
}
```

User-defined Custom Exception in Java

```
// A Class that represents use-defined exception
class MyException extends Exception
{
    public MyException(String s)
    {
        // Call constructor of parent Exception
        super(s);
    }
}

// A Class that uses above MyException
public class Main
{
    // Driver Program

    public static void main(String args[])
    {
        try
        {
            // Throw an object of user defined
            throw new MyException("Hello");
        }
        catch (MyException ex)
        {
            System.out.println("Caught");

            // Print the message from MyException
            System.out.println(ex.getMessage());
        }
    }
}
```

exception

object
Output
Caught
Hello

Topic 1
Types of Exceptions

Topic 2
Exception Methods

Topic 3
Exception Hierarchy

Topic 4
Built in Exceptions

Topic 5
Custom Exceptions

Second Lesson Summary

Understood and Use different types of built in Exceptions and custom exceptions

Course Progress (use before starting new lesson)



Lesson 1. Exceptions



Lesson 2. Built in and Custom Exceptions



Lesson 3. Byte Streams



Lesson 4. Character Streams

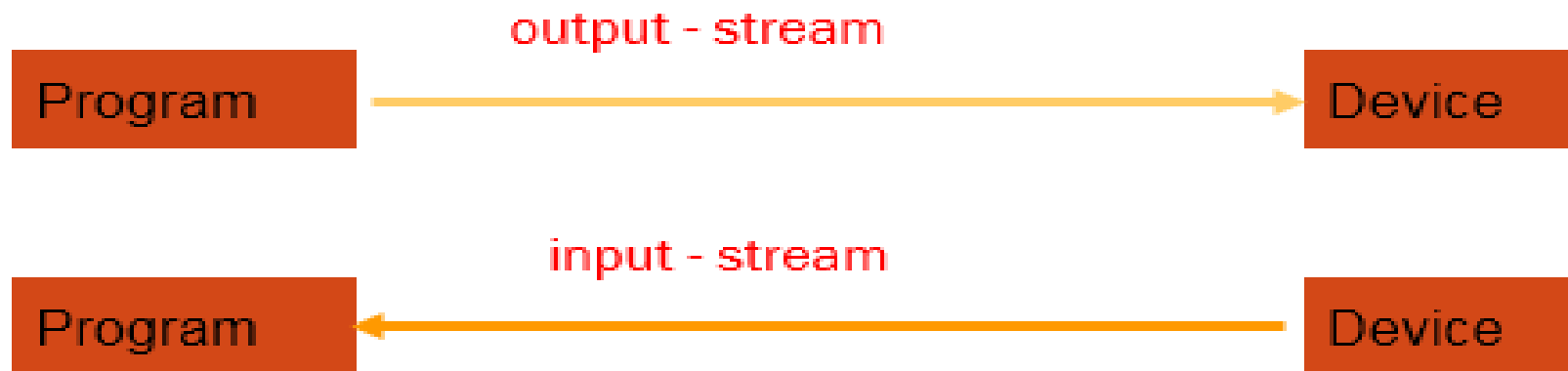


Lesson 5. Abstract and Final

I/O



- Usual Purpose: storing data to ‘nonvolatile’ devices, e.g. harddisk
- Classes provided by package java.io
- Data is transferred to devices by ‘streams’

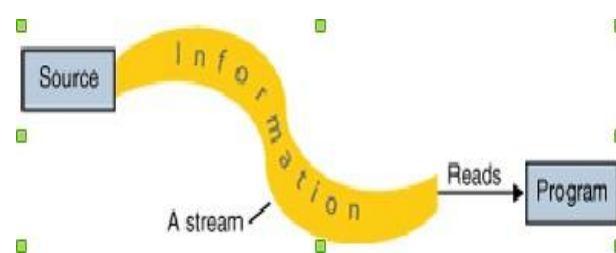


Stream

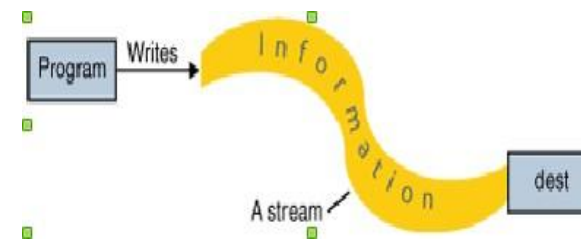
Java programs perform I/O through streams.

- **Stream:** is an abstraction that either produces or consumes information. A stream is linked to a physical device by the Java I/O system.
- **Input stream** can abstract many different kinds of input: **from a disk file, a keyboard, or a network socket.**
- **Output stream** may refer to the **console, a disk file, or a network connection.**

Java implements streams within class hierarchies defined in the **java.io** package.



Input stream



Output stream

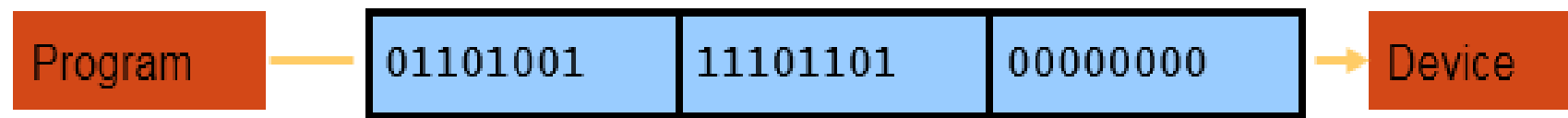
Overview of I/O streams

- The java.io package contains a collection of stream classes that support algorithms for reading and writing.
- To use these classes, a program needs to import the java.io package.
- The stream classes are divided into two class hierarchies, based on the data type (either characters or bytes) on which they operate
 - i.e Character Stream Classes
 - Byte Stream Classes

JAVA distinguishes between 2 types of streams:



- Binary Streams, containing 8 - bit information



I/O Streams Having



I/O Streams Having

- 2 types of streams (text / binary)
- 2 directions (input / output)

Results in 4 base-classes dealing with I/O:

1. InputStream: byte-input
2. OutputStream: byte-output
3. Reader: text-input
4. Writer: text-output

Streams

Java defines two types of streams: **byte** and **character**.

- **Byte streams:** provide a convenient means for handling input and output of bytes. Byte streams are used, for example, when reading or writing binary data.
- **Character streams:** provide a convenient means for handling input and output of characters. They use Unicode and, therefore, can be internationalized. Also, in some cases, character streams are more efficient than byte streams.
- **Byte streams** are defined by using two class hierarchies. **InputStream** and **OutputStream**.
- **Character streams** are defined by using two class hierarchies. **Reader** and **Writer**.

Byte Stream Classes

Stream Class	Meaning
BufferedInputStream	Buffered input stream
BufferedOutputStream	Buffered output stream
ByteArrayInputStream	Input stream that reads from a byte array
ByteArrayOutputStream	Output stream that writes to a byte array
DataInputStream	An input stream that contains methods for reading the Java standard data types
DataOutputStream	An output stream that contains methods for writing the Java standard data types
FileInputStream	Input stream that reads from a file
FileOutputStream	Output stream that writes to a file
FilterInputStream	Implements InputStream
FilterOutputStream	Implements OutputStream
InputStream	Abstract class that describes stream input
ObjectInputStream	Input stream for objects
ObjectOutputStream	Output stream for objects
OutputStream	Abstract class that describes stream output
PipedInputStream	Input pipe
PipedOutputStream	Output pipe
PrintStream	Output stream that contains print() and println()
PushbackInputStream	Input stream that supports one-byte “unget,” which returns a byte to the input stream
SequenceInputStream	Input stream that is a combination of two or more input streams that will be read sequentially, one after the other

InputStream vs OutputStream



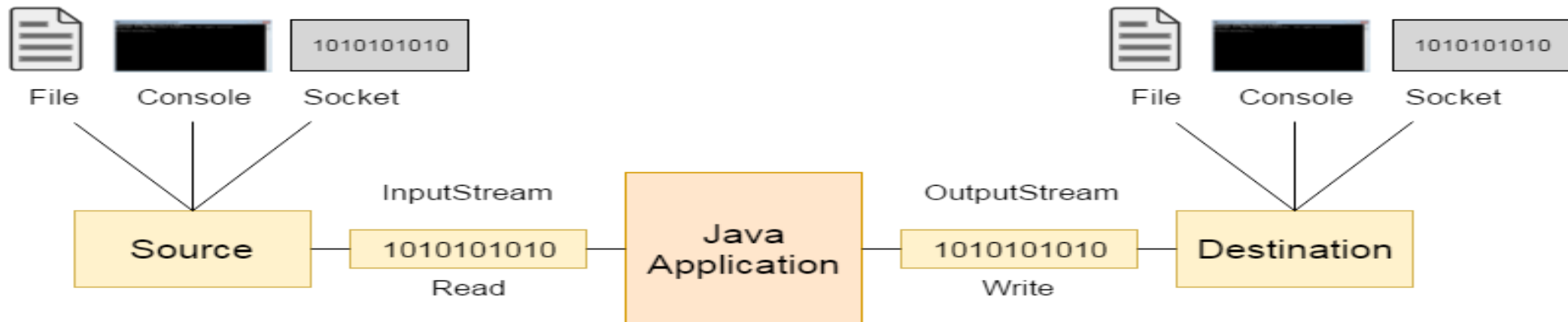
InputStream

Java application uses an input stream to read data from a source, it may be a file, an array, peripheral device or socket.

OutputStream

Java application uses an output stream to write data to a destination, it may be a file, an array, peripheral device or socket.

Let's understand working of Java OutputStream and InputStream by the figure given below.



java.lang

- Object
 - InputStream
 - File
 - FilenameFilter
 - FileDescriptor
 - RandomAccessFile
 - OutputStream
 - ObjectStreamClass
 - StreamTokenizer
 - Reader
 - Writer
 - Serializable
 - Externalizable

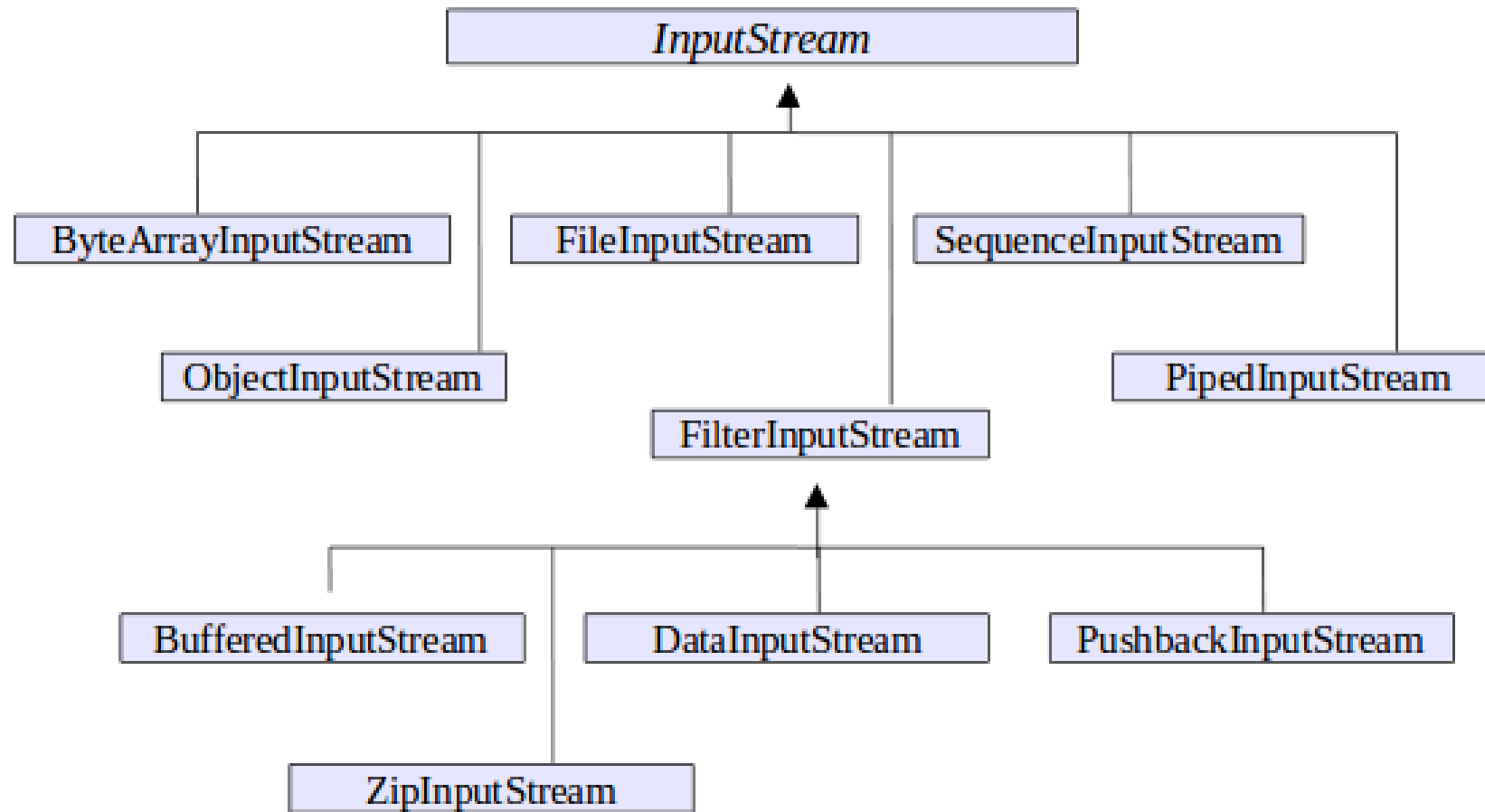
java.io

- InputStream
 - ByteArrayInputStream
 - FileInputStream
 - FilterInputStream
 - ObjectInputStream
 - DataInput
 - ObjectInput
 - DataOutput
 - ObjectOutput
 - PipedInputStream
 - SequenceInputStream
 - StringBufferInputStream
 - OutputStream
 - ByteArrayOutputStream
 - FileOutputStream
 - FilterOutputStream
 - ObjectOutputStream
 - DataOutput
 - ObjectOutput
 - DataOutput
 - ObjectOutput
 - PipedOutputStream
 - Reader
 - BufferedReader
 - LineNumberReader
 - CharArrayReader
 - FilterReader
 - PushbackReader
 - InputStreamReader
 - FileReader
 - PipedReader
 - StringReader
 - Writer
 - BufferedWriter
 - FileWriter
 - CharArrayWriter
 - FilterWriter
 - OutputStreamWriter
 - FileWriter
 - PipedWriter
 - PrintWriter
 - StringWriter

KEY

 - CLASS (rectangle)
 - INTERFACE (rounded rectangle)
 - ABSTRACT CLASS (parallelogram)
 - FINAL CLASS (hexagon)
 - DEPRECATED CLASS (dashed border)
 - extends
 - - - implements

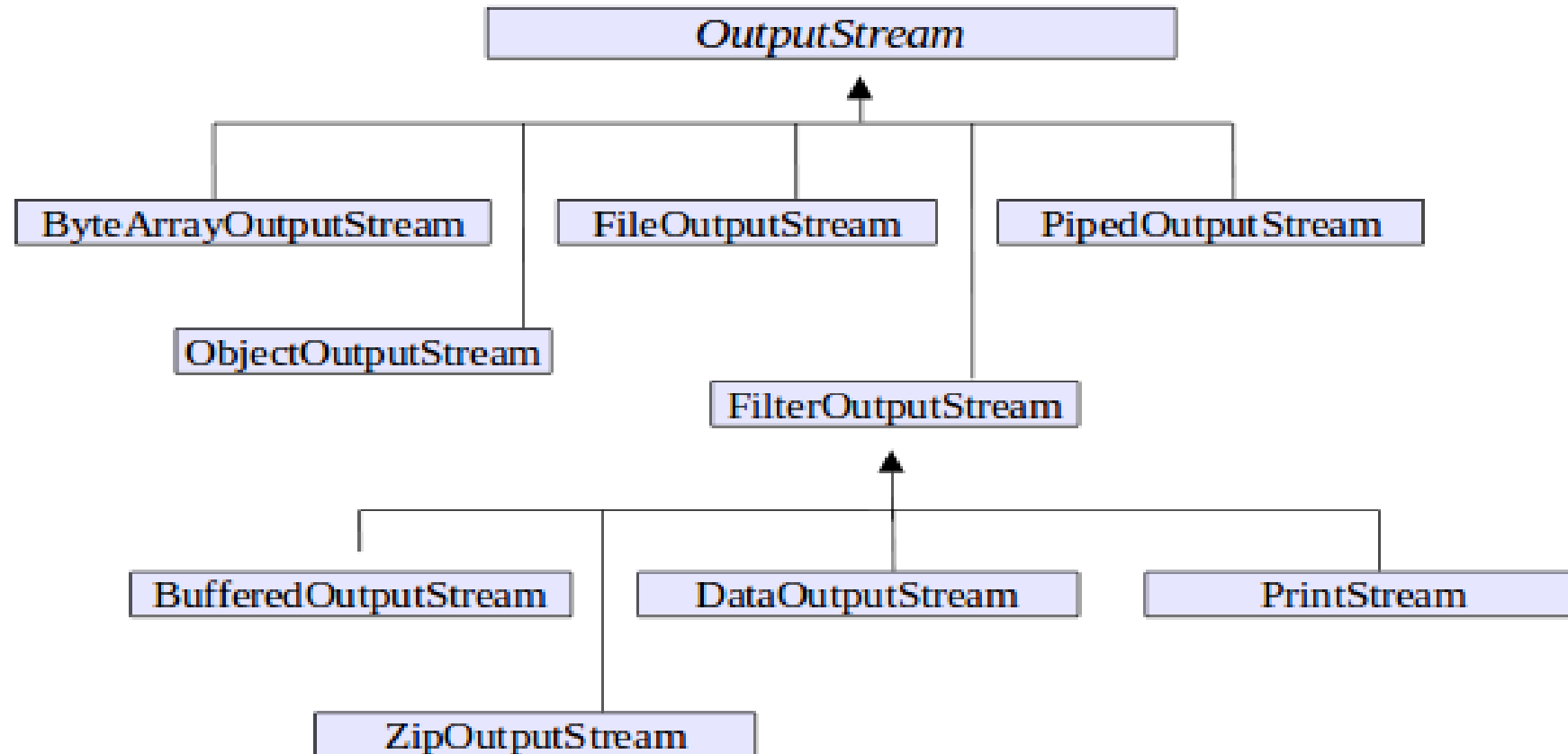
Byte Stream Classes-Input Stream



Common Method in InputStream

Method	Description
<code>public abstract <u>int</u> read() throws IOException</code>	reads the next byte of data from the input stream. It returns -1 at the end of file.
<code>public <u>int</u> available() throws IOException</code>	returns an estimate of the number of bytes that can be read from the current input stream.
<code>public void close() throws IOException</code>	is used to close the current input stream.

Byte Stream Classes-Output Stream



Common Method in OutputStream

Method	Description
1) <code>public void write(int) throws IOException</code>	is used to write a byte to the current output stream.
2) <code>public void write(byte[]) throws IOException</code>	is used to write an array of byte to the current output stream.
3) <code>public void flush() throws IOException</code>	flushes the current output stream.
4) <code>public void close() throws IOException</code>	is used to close the current output stream.

Methods in InputStream



- 1 int available() This method returns an estimate of the number of bytes that can be read (or skipped over) from this input stream without blocking by the next invocation of a method for this input stream.
- 2 void close() This method closes this input stream and releases any system resources associated with the stream.
- 3 void mark(int readlimit) This method marks the current position in this input stream.
- 4 boolean markSupported() This method tests if this input stream supports the mark and reset methods.
- 5 abstract int read() This method reads the next byte of data from the input stream.

Methods in InputStream Con"

6 int read(byte[] b) This method reads some number of bytes from the input stream and stores them into the buffer array b

7 int read(byte[] b, int off, int len) This method reads up to len bytes of data from the input stream into an array of bytes.

8 void reset() This method repositions this stream to the position at the time the mark method was last called on this input stream.

9 long skip(long n) This method skips over and discards n bytes of data from this input stream.

Methods in OutputStream

- 1 void close() This method closes this output stream and releases any system resources associated with this stream.
- 2 void flush() This method flushes this output stream and forces any buffered output bytes to be written out.
- 3 void write(byte[] b) This method writes *b.length* bytes from the specified byte array to this output stream.
- 4 void write(byte[] b, int off, int len) This method writes *len* bytes from the specified byte array starting at offset *off* to this output stream.
- 5 abstract void write(int b) This method writes the specified byte to this output stream.

BufferedInputStream Example

```
// Java program to demonstrate working of BufferedInputStream
import java.io.BufferedInputStream;
import java.io.FileInputStream;
import java.io.IOException;

// Java program to demonstrate BufferedInputStream methods
class BufferedInputStreamDemo
{
    public static void main(String args[]) throws IOException
    {
        // attach the file to FileInputStream
        FileInputStream fin = new FileInputStream("file1.txt");

        BufferedInputStream bin = new BufferedInputStream(fin);

        // illustrating available method
        System.out.println("Number of remaining bytes:" + bin.available());

        // illustrating markSupported() and mark() method
        boolean b=bin.markSupported();
        if (b)
            bin.mark(bin.available());
        // illustrating skip method

        /*Original File content:
        * This is my first line
        * This is my second line*/
        bin.skip(4);
```

```
// illustrating skip method
/*Original File content:
* This is my first line
* This is my second line*/
bin.skip(4);
System.out.println("FileContents :");

// read characters from FileInputStream and
// write them
int ch;
while ((ch=bin.read()) != -1)
    System.out.print((char)ch);

// illustrating reset() method
bin.reset();
while ((ch=bin.read()) != -1)
    System.out.print((char)ch);

// close the file
fin.close(); } }
```

Output: Number of remaining bytes:47

FileContents : is my first line This is my second line This is my first line This is my second line

BufferedOutputStream Example

```
//Java program demonstrating BufferedOutputStream

import java.io.*;

class BufferedOutputStreamDemo
{
    public static void main(String args[])throws Exception
    {
        FileOutputStream fout = new FileOutputStream("f1.txt");

        //creating bufferedOutputStream obj
        BufferedOutputStream bout = new BufferedOutputStream(fout);

        //illustrating write() method
        for(int i = 65; i < 75; i++)

        {
            bout.write(i);
        }

        byte b[] = { 75, 76, 77, 78, 79, 80 };
        bout.write(b);

        //illustrating flush() method
        bout.flush();

        //illustrating close() method
        bout.close();
        fout.close();
    } }
```

Output

ABCDEFGHIJKLMNOP

ByteArrayInputStream

The ByteArrayInputStream is composed of two words: ByteArray and InputStream.

As the name suggests, it can be used to read byte array as input stream.

Java ByteArrayInputStream class contains an internal buffer which is used to **read byte array** as stream.

In this stream, the data is read from a byte array.

The buffer of ByteArrayInputStream automatically grows according to data.

Java ByteArrayInputStream class constructors



Constructor	Description
<code>ByteArrayInputStream(byte[] ary)</code>	Creates a new byte array input stream which uses ary as its buffer array.
<code>ByteArrayInputStream(byte[] ary, int offset, int len)</code>	Creates a new byte array input stream which uses ary as its buffer array that can read up to specified len bytes of data from an array.

```
import java.io.*;

public class ReadExample
{
    public static void main(String[] args) throws IOException
    {
        byte[] buf = { 35, 36, 37, 38 };           // Create the new byte array input stream
        ByteArrayInputStream byt = new ByteArrayInputStream(buf);
        int k = 0;
        while ((k = byt.read()) != -1)
        {
            //Conversion of a byte into character
            char ch = (char) k;
            System.out.println("ASCII value of Character is:" + k + "; Special character is: " + ch);
        }
    }
}
```

Topic 1
Streams

Topic 2
Byte Streams

Topic 3
Buffered Stream

Third Lesson Summary

Use Byte streams for reading and writing

Course Progress (use before starting new lesson)



Lesson 1. Exceptions



Lesson 2. Built in and Custom Exceptions



Lesson 3. Byte Stream



Lesson 4. Character Stream

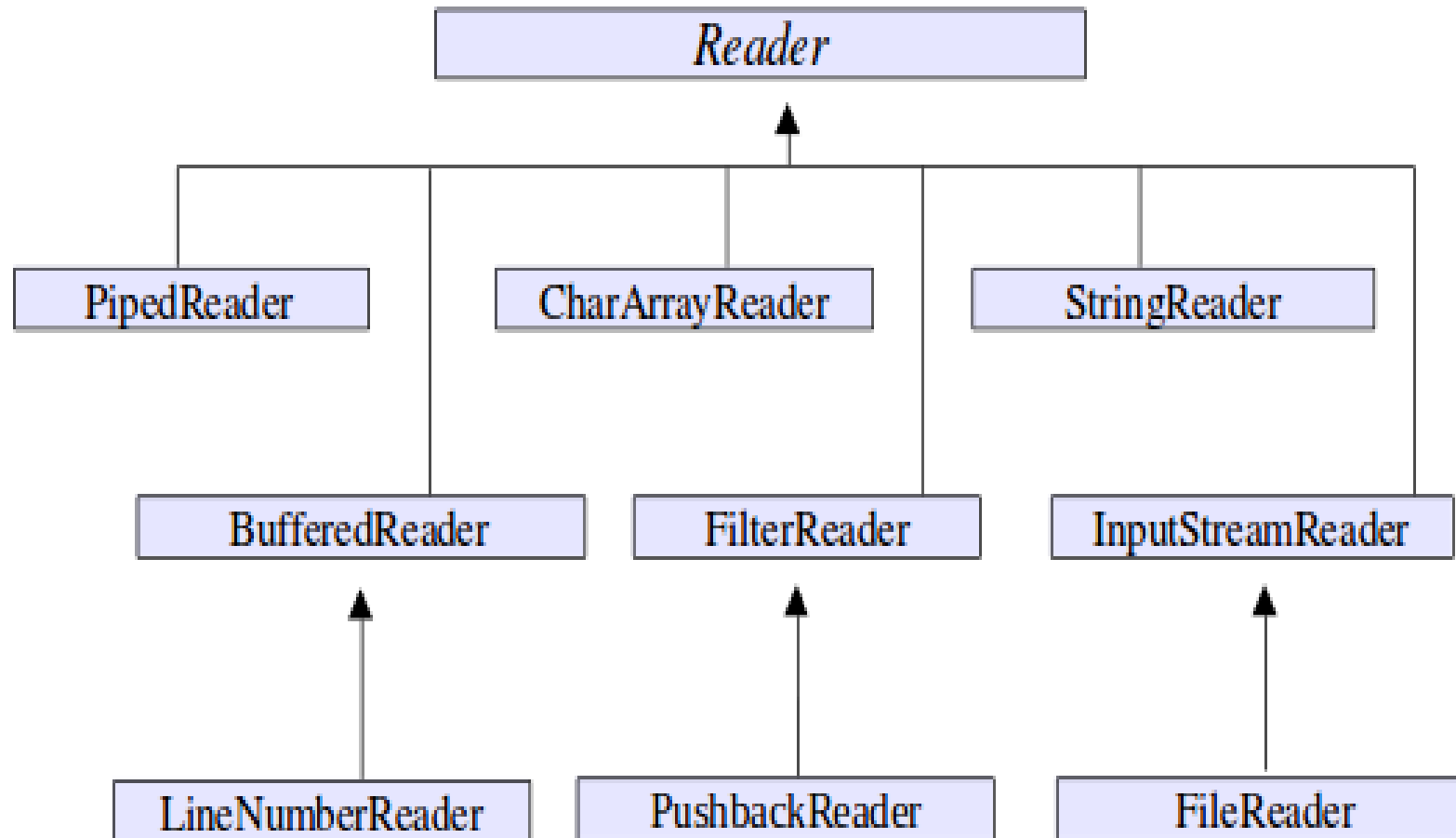


Lesson 5. Abstract and Final

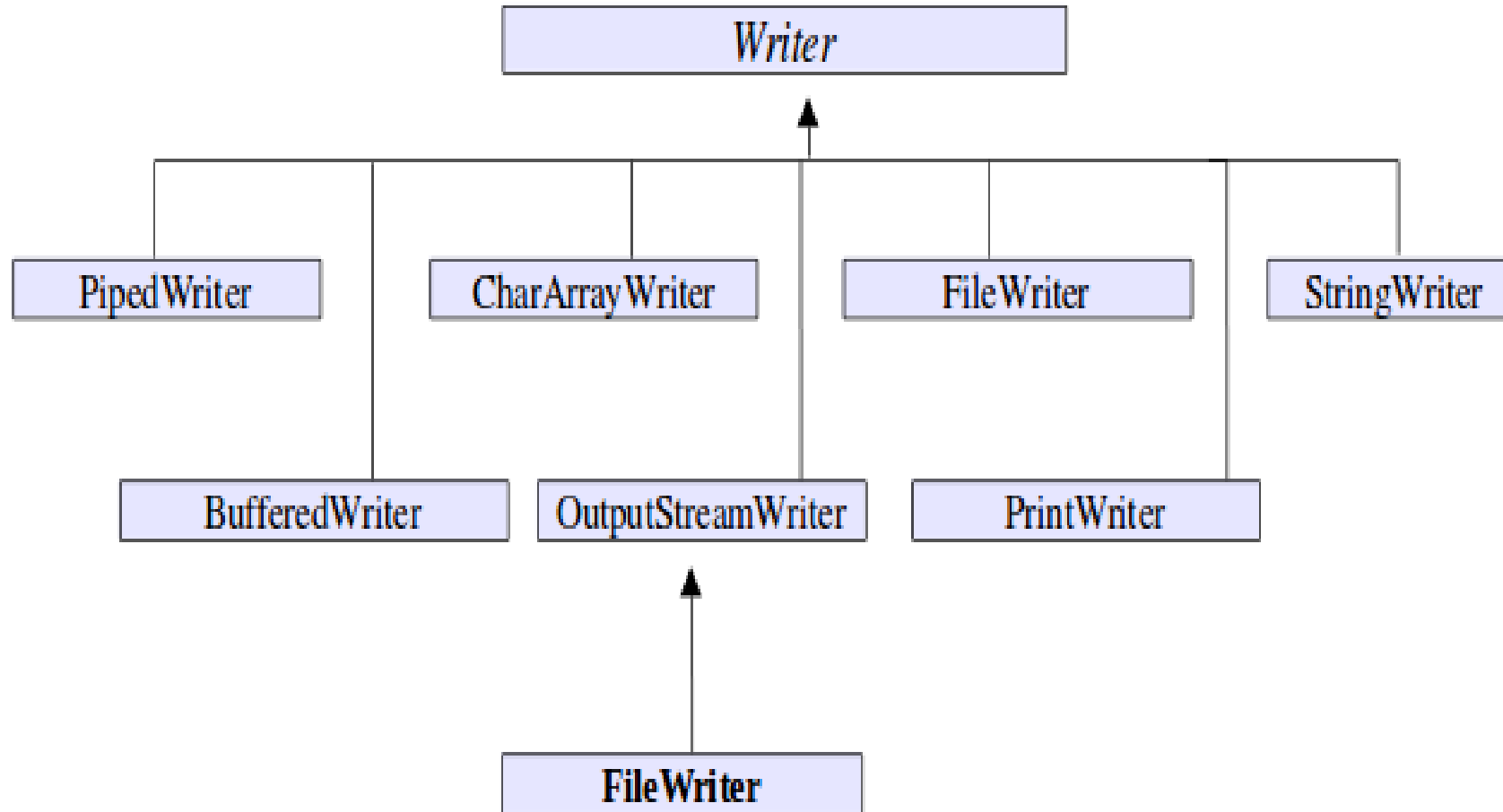
Character Streams

Stream Class	Meaning
BufferedReader	Buffered input character stream
BufferedWriter	Buffered output character stream
CharArrayReader	Input stream that reads from a character array
CharArrayWriter	Output stream that writes to a character array
FileReader	Input stream that reads from a file
FileWriter	Output stream that writes to a file
FilterReader	Filtered reader
FilterWriter	Filtered writer
InputStreamReader	Input stream that translates bytes to characters
LineNumberReader	Input stream that counts lines
OutputStreamWriter	Output stream that translates characters to bytes
PipedReader	Input pipe
PipedWriter	Output pipe
PrintWriter	Output stream that contains print() and println()
PushbackReader	Input stream that allows characters to be returned to the input stream
Reader	Abstract class that describes character stream input
StringReader	Input stream that reads from a string
StringWriter	Output stream that writes to a string
Writer	Abstract class that describes character stream output

Character Stream Classes: Reader



Character Stream Classes: Writer



BufferedInputStream

Java BufferedInputStream class is used to read information from stream.

It internally uses buffer mechanism to make the performance fast.

The important points about BufferedInputStream are:

- When the bytes from the stream are skipped or read, the internal buffer automatically refilled from the contained input stream, many bytes at a time.
- When a BufferedInputStream is created, an internal buffer array is created.

Methods



Method	Description
<code>int available()</code>	It returns an estimate number of bytes that can be read from the input stream without blocking by the next invocation method for the input stream.
<code>int read()</code>	It read the next byte of data from the input stream.
<code>int read(byte[] b, int off, int ln)</code>	It read the bytes from the specified byte-input stream into a specified byte array, starting with the given offset.
<code>void close()</code>	It closes the input stream and releases any of the system resources associated with the stream.
<code>void reset()</code>	It repositions the stream at a position the mark method was last called on this input stream.
<code>void mark(int readlimit)</code>	It sees the general contract of the mark method for the input stream.
<code>long skip(long x)</code>	It skips over and discards x bytes of data from the input stream.
<code>boolean markSupported()</code>	It tests for the input stream to support the mark and reset methods.

Java BufferedOutputStream Class



Java BufferedOutputStream class is used for buffering an output stream.

It internally uses buffer to store data.

It adds more efficiency than to write data directly into a stream. So, it makes the performance fast.

For adding the buffer in an OutputStream, use the BufferedOutputStream class.

Let's see the syntax for adding the buffer in an OutputStream:

```
OutputStream os= new BufferedOutputStream(new FileOutputStream("D:\\IO Package\\test  
out.txt"));
```

Reading Console Input



- Console input is accomplished by reading from **System.in**.
- To obtain a character-based stream that is attached to the console, wrap **System.in** in a **BufferedReader** object. **BufferedReader** supports a buffered input stream.
- A commonly used constructor is shown here:
BufferedReader(Reader inputReader)
- Here, **inputReader** is the stream that is linked to the instance of **BufferedReader** that is being created.
- **Reader** is an abstract class. One of its concrete subclasses is **InputStreamReader**, which converts bytes to characters.

Reading Console Input

- To obtain an `InputStreamReader` object that is linked to `System.in`, use the following constructor:

`InputStreamReader(InputStream inputStream)`

- Because `System.in` refers to an object of type `InputStream`, it can be used for `inputStream`.
- The following line of code creates a `BufferedReader` that is connected to the keyboard:

`BufferedReader br = new BufferedReader(new InputStreamReader(System.in));`

- After this statement executes, `br` is a character-based stream that is linked to the console through `System.in`.

Using BufferedReader to read characters from the console

```
import java. io .*;
class BRRead {
public static void main( String args []) throws IOException
{
char c;

Buffered Reader br = new Buffered Reader ( new InputStream Reader ( System . in));

System . out. println (" Enter characters , ' q' to quit.");
do {

c = ( char) br. read (); System . out. println ( c);
} while( c != ' q');
} }
```

Read a string from console using a BufferedReader

```
import java. io .*;
class BRReadLines {
public static void main( String args []) throws IOException
{
Buffered Reader br = new Buffered Reader ( new InputStream Reader ( System . in ));
String str;
System . out. println (" Enter lines of text.");
System . out. println (" Enter ' stop ' to quit.");
do {
str = br. readLine (); System . out. println ( str );
} while (! str. equals(" stop"));
} }
```

Read a number from console using a BufferedReader

```
import java. io .*;
public class UserInputInteger
{
public static void main( String args []) throws IOException
{
InputStream Reader read = new InputStream Reader ( System . in );
BufferedReader in = new BufferedReader ( read );
int number;

System . out. println (" Enter the number");
number = Integer. parseInt( in. readLine ());

System . out. println ( number );
}
}
```

Writing Console Output

```
class WriteDemo {  
    public static void main( String args [])  
    {  
        int b;  
        b = ' A';  
  
        System . out. write( b);  
  
        System . out. write('\ n');  
    }  
}
```

```
import java. io .*;  
public class PrintWriterDemo {  
    public static void main( String args []) {  
        PrintWriter pw = new PrintWriter ( System . out .true );  
        pw. println (" This is a string ");  
        int i = -7;  
        pw. println (i);  
        double d = 4.5 e -7;  
        pw. println ( d);  
    }  
}
```

Reading and Writing Files

- Two of the most often-used stream classes are **FileInputStream** and **FileOutputStream**, which create byte streams linked to files.
- To open a file, you simply create an object of one of these classes, specifying the name of the file as an argument to the constructor.
- Although both classes support additional constructors, the following are the forms that we will be using:

FileInputStream(String fileName) throws FileNotFoundException

FileOutputStream(String fileName) throws FileNotFoundException

Reading and Writing Files - Example

```
import java. io .*;
class CopyFile {

public static void main( String args []) throws IOException
{

int i; FileInputStream fin;

FileOutputStream fout;

try {
// open input file
try {
} fin = new FileInputStream ( args [0]);
catch ( FileNotFoundException e)
{ System . out. println (" Input File Not Found ");
return ; }
```

Reading and Writing Files – Example Contd.

```
try {  
    fout = new FileOutputStream ( args [1]);  
  
} catch ( FileNotFoundException e) { System . out. println (" Output File Error"); return ;  
}  
  
} catch ( Array Index Out OfBounds Exception e)  
{ System . out. println (" Usage: CopyFile From To");  
return ;}  
try {  
    do {  
        i = fin. read ();  
        if( i != -1) fout. write( i);  
        } while( i != -1);  
  
    } catch ( IOException e) { System . out. println (" File Error");  
    }  
    fin. close ();  
    fout. close ();  
}}
```

```
import java.io.FileInputStream;

public class DataStreamExample
{
    public static void main(String args[])
    {
        try
        {
            FileInputStream fin=new FileInputStream("D:\\testout.txt");

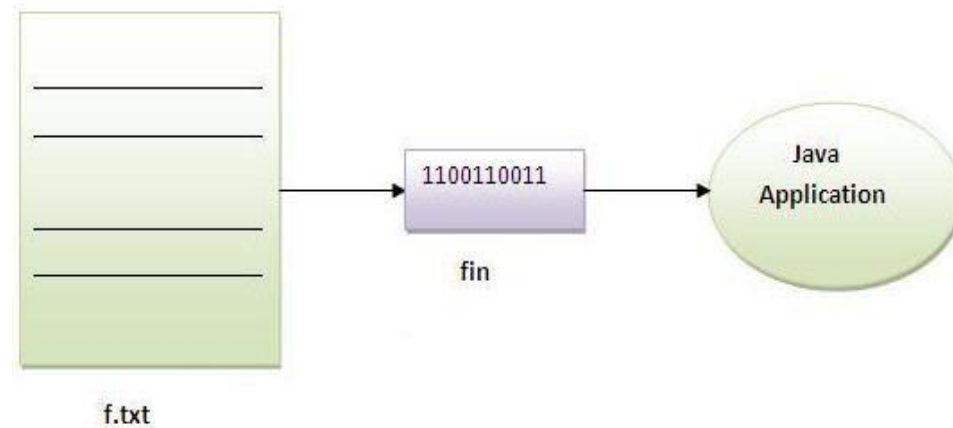
            int i=fin.read();

            System.out.print((char)i);

            fin.close();

        }
        catch(Exception e)
        {
            System.out.println(e);

        }
    }
}
```



Note: Before running the code, a text file named as "**testout.txt**" is required to be created. In this file, we are having following content:

Hello welcomes you all for java programming.

After executing the above program, you will get a single character from the file which is 75 (in byte form).

To see the text, you need to convert it into character.

Output:

H

```
import java.io.*;
class SimpleRead
{
    public static void main(String args[])
    {
        try
        {
            FileInputStream fin=new FileInputStream("abc.txt");
            int i=0;
            while((i=fin.read())!=-1)
            {
                System.out.println((char)i);
            }
            fin.close();
        }
        catch(Exception e)    { system.out.println(e);    } } }
```

Output:
**Hello welcomes you all for java
programming.**

Topic 1
Character Streams

Topic 2
Reading & Writing Console I/O

Topic 3
BufferedReader

Topic 4
Reading and Writing Files

Fourth Lesson Summary

Use BufferedReader, Character and File Streams for reading and writing

Course Progress (use before starting new lesson)



Lesson 1. Exceptions



Lesson 2. Built in and Custom Exceptions



Lesson 3. Byte Stream



Lesson 4. Character Stream



Lesson 5. Abstract and Final

Abstract methods

- you can declare a *method* without defining it:

```
public abstract void draw(int size);
```

 - Notice that the body of the method is missing
- A method that has been declared but not defined is an abstract method

Abstract classes I

- Any class containing an abstract method is an abstract class
- You must declare the class with the keyword `abstract`:

```
abstract class MyClass {...}
```

- An abstract class is *incomplete*
 - It has “missing” method bodies
- You cannot instantiate (create a new instance of) an abstract class

Abstract classes II

- You can extend (subclass) an abstract class
 - If the subclass defines all the inherited abstract methods, it is “complete” and can be instantiated
 - If the subclass does *not* define all the inherited abstract methods, it too must be abstract
- You can declare a class to be abstract even if it does not contain any abstract methods
 - This prevents the class from being instantiated

Why have abstract classes?

- Suppose you wanted to create a class Shape, with subclasses Oval, Rectangle, Triangle, Hexagon, etc.
- You don't want to allow creation of a "Shape"
 - Only *particular* shapes make sense, not *generic* ones
 - If Shape is abstract, you can't create a new Shape
 - You *can* create a new Oval, a new Rectangle, etc.
- Abstract classes are good for defining a general category containing specific, "concrete" classes

An example abstract class

- public abstract class Animal
- {
 abstract int eat();
 abstract void breathe();
}
- This class cannot be instantiated
- Any non-abstract subclass of Animal must provide the eat() and breathe() methods

EXAMPLE

- `class Shape { ... }`
- `class Star extends Shape {
 void draw() { ... }
 ...
}`
- `class Crescent extends Shape {
 void draw() { ... }
 ...
}`
- `Shape someShape = new Star();`
 - This is legal, because a Star *is* a Shape
- `someShape.draw();`
 - This is a syntax error, because *some* Shape might not have a `draw()` method
 - Remember: ***A class knows its superclass, but not its subclasses***

Solution

- abstract class Shape {
 void draw();
}
- class Star extends Shape {
 void draw() { ... }
 ...
}
- class Crescent extends Shape {
 void draw() { ... }
 ...
}
- Shape someShape = new Star();
 - This is legal, because a Star *is* a Shape
 - However, Shape someShape = new Shape(); is *no longer* legal
- someShape.draw();
 - This is legal, because every actual instance *must* have a draw() method

Final Keyword



- The **final keyword** in java is used to restrict the user.
- The java final keyword can be used in many context.
- Final can be:
 - variable
 - method
 - class

- The final keyword can be applied with the variables, a final variable that have no value it is called blank final variable or uninitialized final variable.
- It can be initialized in the constructor only.
- The blank final variable can be static also which will be initialized in the static block only.
- Let's first learn the basics of final keyword.

Java final variable

- If you make any variable as final, you cannot change the value of final variable(It will be constant).
- **Example of final variable**
 - There is a final variable speed limit, we are going to change the value of this variable, but It can't be changed because final variable once assigned a value can never be changed.

EXAMPLE

```
class Bike9
{
    final int speedlimit=90; //final variable
    void run()
    {
        speedlimit=400;
    }
    public static void main(String args[])
    {
        Bike9 obj=new Bike9();
        obj.run();
    }
} //end of class
```

Java final method

- If you make any method as final, you cannot override it.

```
class Bike
```

```
{  
    final void run()  
    {  
        System.out.println("running");  
    }  
}
```

```
class Honda extends Bike
```

```
{  
    void run()  
    {System.out.println("running safely with 100kmph");}  
    public static void main(String args[])  
    {  
        Honda honda= new Honda();  
        honda.run();  
    }  
}
```

Java final class

- If you make any class as final, you cannot extend it.

final class Bike

```
{  
}
```

class Honda1 extends Bike

```
{
```

```
    void run()  
{
```

```
    {
```

```
        System.out.println("running safely with 100kmph");
```

```
    }
```

```
    public static void main(String args[])
```

```
    {
```

```
        Honda1 honda= new Honda();
```

```
        honda.run();
```

```
    }
```

```
}
```

Is final method inherited?

- Yes, final method is inherited but you cannot override it.

```
class Bike
```

```
{
```

```
    final void run()
```

```
{
```

```
        System.out.println("running...");
```

```
}
```

```
}
```

```
class Honda2 extends Bike
```

```
{
```

```
    public static void main(String args[])
```

```
{
```

```
        new Honda2().run();
```

```
}
```

```
}
```

What is blank or uninitialized final variable?



- A final variable that is not initialized at the time of declaration is known as blank final variable.
- If you want to create a variable that is initialized at the time of creating object and once initialized may not be changed, it is useful.
- For example PAN CARD number of an employee.
- It can be initialized only in constructor.

Can we initialize blank final variable?

Yes, but only in constructor.

```
class Bike10
{
    final int speedlimit; // blank final variable
    Bike10()
    {
        speedlimit=70;
        System.out.println(speedlimit);
    }
    public static void main(String args[])
    {
        new Bike10();
    }
}
```

static blank final variable

- A static final variable that is not initialized at the time of declaration is known as static blank final variable.
- It can be initialized only in static block.

```
class A
{
    static final int data; //static blank final variable
    static
    {
        data=50;
    }
    public static void main(String args[])
    {
        System.out.println(A.data);
    }
}
```

What is final parameter?

- If you declare any parameter as final, you cannot change the value of it.

```
class Bike11
{
    int cube(final int n)
    {
        n=n+2;//can't be changed as n is final
        n*n*n;
    }
    public static void main(String args[])
    {
        Bike11 b=new Bike11();
        b.cube(5);
    }
}
```

Can we declare a constructor final?

- No, because constructor is never inherited.

Topic 1
Abstract class

Topic 2
final keyword

Fifth Lesson Summary

Working of abstract and final keyword

Thank You!